

Chat Agent — Luồng và xử lý chi tiết

YoMedia Dashboard · NovaAI Assistant

Tài liệu nội bộ · Cập nhật 2026-06-08 · Nguồn: docs/chat-agent-flow.md · Build Demo 4 lớp: server-build-demo-architecture.md · PDF: chat-agent-flow.pdf

1. Tổng quan

Hệ thống chat AI: **request/response qua tRPC** (không streaming). Kiến trúc **Supervisor → Router → Agents → Composer**.

Lớp	File	Vai trò
Client	apps/web/components/Chat.tsx	UI, localStorage, provider, file/folder
API	api.rag.query	Mutation có auth (Clerk Bearer)
Controller	controllers/trpc/ragRouter.ts	Zod + assertChatAccess(/chat)
Entry	answerWithRag	Wrapper gọi runSupervisor (+ req)
Supervisor	runSupervisor.ts	Guardrails, routing, agents, compose, log

Lưu ý: Bubble UI (localStorage) ≠ ngữ cảnh LLM (server RAM). Refresh/xóa tin local không xóa short memory — dùng **Xóa phiên** hoặc rag.clearSession.

2. Luồng tổng thể

```
User → Chat.tsx → localStorage (bubble user)
  → readFileAsDataURL → base64 (nếu có file/folder)
  → api.rag.query(question, provider, attachments, sessionId)
  → controllers/trpc/ragRouter (protected + assertChatAccess)
  → answerWithRag → runSupervisor(req)
  → guardrails → getShortMemory + mergeBuildDemoAttachments
  → resolveRoute → agents[] (1 hoặc nhiều)
  → runAgents (Promise.all – song song)
  → composeMultiResponse → appendShortMemoryTurn + logChatEvent
  → Response → bubble assistant (+ BuildDemoProgress nếu UI)
```

sessionId server = activeConversation.id client (vd. c-1717...).

2.4 Flow Chat RAG chi tiết (4 lớp controllers)

Sequence end-to-end

1. User → Chat.tsx (nhập câu hỏi, optional file)

2. Chat.tsx → localStorage (bubble user – chỉ UI)

3. Chat.tsx → readFileAsDataURL → attachments base64

4. api.rag.query → POST /api/trpc/rag.query + Authorization Bearer

5. trpc/context.ts → createContext → ctx.auth (Clerk)

6. controllers/trpc/ragRouter → protectedProcedure

7. shared/schemas/rag.schema → Zod validate input

8. authPolicy.service → assertChatAccess(/chat)

9. answerWithRag → runSupervisor

10. guardrails → shortMemory (key = email+role+sessionId)

11. resolveRoute → nếu rag: runRagAgent

12. retrieveKnowledgeContext (apps/server/rag/docs)

13. callProvider (Gemini/OpenAI) + history

14. composeMultiResponse → appendShortMemoryTurn → aiLogger

15. Response → Chat.tsx bubble assistant → localStorage

Kiến trúc 4 lớp

Lớp	Thư mục	Vai trò
L1 Controller	controllers/trpc/ , controllers/rest/ , controllers/ai/	Zod, auth, gọi orchestrator/service
L2 Service	services/ , lib/ai/orchestration/	Supervisor, policy, business logic
L3 Repository	repositories/	Đọc/ghi JSON (brands, creative-demos)
L4 Shared	shared/schemas/	Zod schema dùng chung

Client (Chat.tsx → api.rag.query)

↓ POST /api/trpc

trpc/appRouter → controllers/trpc/ragRouter

↓ assertChatAccess + answerWithRag

runSupervisor → resolveRoute → runRagAgent

↓

knowledgeBase (rag/docs) → callProvider → responseComposer

↓

{ answer, intent, sources, provider, fallbackUsed }

Nhánh RAG — retrieval → LLM

question

→ load apps/server/rag/docs/*.md|txt|json (cache in-memory)

- questionKeywords + scoreText trên từng doc
- top 3 doc → extract snippet (~2400 chars/doc)
- ghép contextPrompt
- callProvider(provider, prompt, history từ shortMemory)
- nếu primary fail → fallback openai ↔ gemini
- AgentResult { answer, sources[], spans }

Khi nào route vào RAG

Điều kiện	Kết quả
Tool Build Demo + candidate rag	Chỉ rag — QA ưu tiên phiên demo
detectAgentCandidates = rag	agents: ["rag"]
LLM / rule → knowledge_qa	agents: ["rag"]
Multi-intent có rag	Chạy song song với agents khác

rag.clearSession

```
Chat.tsx → rag.clearSession({ sessionId }) hoặc { allSessions: true }
  → assertChatAccess(/chat)
  → clearShortMemory(key) hoặc clearShortMemoryByPrefix(email)
  → logChatEvent(chat_clear_history)
  → { ok: true, cleared }
```

Payload & response RAG

Request	Response (RAG)
question (bắt buộc)	answer — câu trả lời LLM
provider gemini openai	intent knowledge_qa multi_intent
sessionId — phân tách memory	sources — tên file doc
attachments — base64 (Build Demo)	fallbackUsed — đổi provider

File map — flow RAG

Bước	File
UI gửi tin	apps/web/components/Chat.tsx
Client API	apps/web/lib/trpc/api.ts
HTTP mount	apps/server/src/index.ts → /api/trpc

Controller	<code>controllers/trpc/ragRouter.ts</code>
Schema	<code>shared/schemas/rag.schema.ts</code>
Policy	<code>services/authPolicy.service.ts</code>
Orchestrator	<code>lib/ai/orchestration/answerWithRag.ts</code>
Supervisor	<code>lib/ai/agents/supervisor/runSupervisor.ts</code>
Router	<code>lib/ai/agents/router/routeIntent.ts</code>
RAG agent	<code>lib/ai/agents/rag/runRagAgent.ts</code>
Retrieval	<code>lib/ai/retrieval/knowledgeBase.ts</code>
LLM	<code>lib/ai/services/llm/callProvider.ts</code>
Memory	<code>lib/ai/memory/shortMemory.ts</code>

3. Pipeline server (runSupervisor)

Thứ tự cố định: **Guardrails** → **Short memory** → **resolveRoute** → **runAgents** → **composeMultiResponse**

3.1 Guardrails

- Tin trống, > 1500 ký tự, từ nhạy cảm (password, api key, credit card, ...)
- Trả lỗi, không gọi AI — `guardrails/index.ts`

3.2 Short memory

- Key: `{email hoặc role:guest}::{sessionId hoặc default}`
- TTL 2h · tối đa 8 cặp turn
- `buildDemoAttachments` — file pending đến khi upload thành công

3.3 Routing (resolveRoute)

Bước	Điều kiện	Kết quả
1a	Tool + candidate <code>rag</code>	Chỉ <code>rag</code> (QA ưu tiên Build Demo)
1b	Tool + candidates khác	<code>actions</code> + candidates (multi_intent)
1c	Chỉ tool	<code>actions</code>
2	<code>detectAgentCandidates > 1</code>	multi_intent

3	Scoring 1 agent	sql / dashboard / rag
4	LLM → rule classifyUserIntent	rag / free_chat / sql / dashboard

3.4 Tool (ưu tiên cao nhất)

Tool	Trigger
help	help, hướng dẫn, trợ giúp
time_now	mấy giờ, time now, giờ hiện tại
compress_demo_assets	build demo, nén, compress, tv, tạo demo...
upload_sftp_demo	upload, sftp, tải lên, gửi file...

Phiên Build Demo active (`isBuildDemoSessionActive`): file pending; history đã bắt đầu demo; assistant hỏi brand/format + user trả lời tiếp.

3.5 Agents

Agent	File	Xử lý
actions	runActionAgent.ts	help, time_now, build demo
rag	runRagAgent.ts	RAG docs → LLM
free_chat	runFreeChatAgent.ts	LLM trực tiếp
sql	runSqlAgent.ts	LLM sinh SQL → MySQL whitelist
dashboard	runDashboardAgent.ts	Activity log → LLM tóm tắt

Chạy **song song** khi multi-intent. Provider lỗi → fallback OpenAI ↔ Gemini.

4. Quyết định routing

```
User message
├─ Guardrail fail → lỗi (không gọi AI)
├─ resolveActionTool + rag candidate → rag agent (ưu tiên QA)
├─ resolveActionTool only → actions agent
├─ detectAgentCandidates > 1 → multi_intent (song song)
├─ Scoring 1 agent → sql / dashboard / rag
├─ LLM / rule intent
│   ├── knowledge_qa → rag
│   ├── sql_query → sql
│   ├── dashboard_insight → dashboard
│   └── free_chat → free_chat
└─ composeMultiResponse + appendShortMemoryTurn + log
```

5. Build Demo

Tool	Mục đích
upload_sftp_demo	Upload SFTP trực tiếp, không nén
compress_demo_assets	Nén ảnh/video rồi upload SFTP

```
runActionAgent → controllers/ai/buildDemoTool.ts
  → assertBuildDemoSftpAllowed(req)
  → mergeBuildDemoAttachments(memoryKey)
  → buildDemoAgent.ts (LLM function calling)
  → buildDemoInputSchema (Zod) + assertBuildDemoBrandPolicy
  → services/buildDemo.service.ts → executeBuildDemo
    → services/buildDemo/upload.ts | compress.ts
    → SFTP /script/demo/{year}/{month}/...
    → Video: vastXml.ts · HTML: inlineImages.ts
  → "Build Demo thành công" → clearBuildDemoAttachments
```

API: `buildDemoProcessing: true` khi `executeBuildDemo` chạy. `Chat.tsx` đọc field này → 100% ~600ms rồi ẩn; heuristic false + field false → ẩn ngay.

6. Client (Chat.tsx)

```
api.rag.query(text || "upload demo", provider, attachments, activeConversation.id)
```

Bộ nhớ	Nơi	Ghi chú
Conversations	localStorage <code>yomedia.chat.conversations.v1</code>	Max 30
Short memory	Server RAM Map	8 turn LLM
Pending files	Server <code>buildDemoAttachments</code>	Đến upload OK

7. RAG

Index: **.md, .txt, .json** trong `apps/server/rag/docs` — score keyword → top 3 → snippet → LLM.

PDF trong `rag/docs` hiện **không** được index.

8. API tRPC

Router: `controllers/trpc/ragRouter.ts` · Mount: `trpc/appRouter.ts`

Procedure	Mô tả
rag.query	Zod + assertChatAccess + answerWithRag
rag.clearSession	Xóa short memory (một session hoặc allSessions)

Output: ok, answer, provider, intent, sources, toolCalled, buildDemoProcessing, fallbackUsed

Intent: actions | knowledge_qa | free_chat | sql_query | dashboard_insight | multi_intent

9. Bản đồ file chi tiết

Client

File	Vai trò
components/Chat.tsx	UI chat, gửi tin, conversation
components/ChatMessageContent.tsx	Render bubble
components/BuildDemoProgress.tsx	Progress bar giả lập
lib/buildDemoChatProgress.ts	Heuristic progress
lib/chatAttachments.ts	Merge folder/file upload
lib/trpc/api.ts	tRPC client

Server — Controllers & Entry (LAYER 1)

File	Vai trò
controllers/trpc/ragRouter.ts	tRPC rag.query, clearSession
controllers/trpc/*.ts	health, auth, user, admin, creative, ...
controllers/rest/*.ts	REST: sftp, upload, fileUpload, smtp
controllers/ai/buildDemoTool.ts	Build Demo: Zod + ACL + service
orchestration/answerWithRag.ts	Entry wrapper (+ req)
agents/supervisor/runSupervisor.ts	Supervisor chính

Server — Routing & Intent

File	Vai trò
------	---------

agents/router/routeIntent.ts	resolveRoute
tools/detectTool.ts	Action tool detection
intent/scoring.ts	Multi-agent scoring
intent/classifyUserIntent.ts	Rule classifier
services/llm/classifyIntent.ts	LLM classifier

Server — Orchestration & Agents

File	Vai trò
orchestration/runAgents.ts	Agent registry + Promise.all
orchestration/responseComposer.ts	Gộp response
orchestration/aggregator.ts	Rank + merge answers
agents/actions/runActionAgent.ts	Tools + Build Demo
agents/rag/runRagAgent.ts	RAG + LLM
agents/freeChat/runFreeChatAgent.ts	Free chat
agents/sql/runSqlAgent.ts	SQL agent
agents/dashboard/runDashboardAgent.ts	Dashboard agent

Server — Build Demo services

File	Vai trò
services/buildDemo.service.ts	Orchestrator + format markdown
services/authPolicy.service.ts	/chat, SFTP, brand ACL
services/buildDemo/upload.ts	Upload SFTP
services/buildDemo/compress.ts	Nén + upload
lib/ai/tools/buildDemo/buildDemoAgent.ts	LLM extract args
tools/mysql/	MySQL query (whitelist)
tools/analytics/	Activity log summary

Server — Core

File	Vai trò
------	---------

core/types.ts	Types chính
core/config.ts	Model, prompt, limits
memory/shortMemory.ts	Short memory + attachments
retrieval/knowledgeBase.ts	RAG indexing
services/llm/callProvider.ts	OpenAI / Gemini
logging/aiLogger.ts	Chat event logging

Cây thư mục AI

```
apps/server/src/lib/ai/
├── agents/
│   ├── actions/runActionAgent.ts
│   ├── dashboard/runDashboardAgent.ts
│   ├── freeChat/runFreeChatAgent.ts
│   ├── rag/runRagAgent.ts
│   ├── router/routeIntent.ts
│   ├── sql/runSqlAgent.ts
│   └── supervisor/runSupervisor.ts
├── core/ (config.ts, types.ts)
├── guardrails/index.ts
├── intent/ (classifyUserIntent, scoring, types)
├── logging/aiLogger.ts
├── memory/shortMemory.ts
├── orchestration/ (aggregator, answerWithRag, responseComposer, runAgents)
├── retrieval/knowledgeBase.ts
├── services/llm/ (callProvider, classifyIntent)
└── tools/ (analytics, buildDemo, mysql, detectTool, index, types)
```

10. Biến môi trường

Biến	Mục đích
OPENAI_API_KEY / GEMINI_API_KEY	Chat + Build Demo agent
OPENAI_CHAT_MODEL / GEMINI_CHAT_MODEL	Model
CHAT_SYSTEM_PROMPT	System prompt
SHORT_MEMORY_TTL_MS	TTL memory (ms)
MYSQL_*	SQL Agent

11. Gỡ lỗi thường gặp

Triệu chứng	Nguyên nhân có thể
AI quên ngữ cảnh sau refresh	UI vs server memory khác session / chưa clear
Mọi câu → Build Demo	Phiên demo active — Xóa phiên
Progress nhưng không upload	UI heuristic true; server buildDemoProcessing false
403 khi build demo	Thiếu /chat, canSftpUploadBinary, hoặc brand
RAG không trả PDF	Chỉ index md/txt/json
Provider lỗi	API key / cả hai provider fail
SQL Agent từ chối	Role hoặc MySQL chưa cấu hình
Multi-intent nhiều section	mergeAgentAnswers ghép ### SQL, ### Dashboard...